

CFD 2025-26 – Lab 03 – solution

The full code for this lab is provided in the notebook `CFDlab03.ipynb`. The notebook also contains an introductory tutorial on direct/iterative solvers and preconditioning for a scalar elliptic problem. In particular, we highlight that the default parameters of a `LinearVariationalSolver` can be extracted from the predefined dictionary `firedrake.petsc.DEFAULT_KSP_PARAMETERS`. Moreover, we point out that all `LinearVariationalSolvers` (at least when using PETSc as a linear algebra backend library, as we do throughout all this course) wrap the same PETSc Krylov-type solver `KSP`.¹ For this reason, if we want to apply a *direct* solver, we need to “tweak” the parameters not to actually perform iterations: this can be done by setting `'ksp_type': 'preonly'`, so that only the preconditioning step is applied, and choosing as a preconditioner the direct solver of interest, via the parameter `pc_type`.

Exercise 1

- 1 Following the lines of the previous lab sessions, we first derive the weak formulation of the problem.

We point out that, since the problem has *fully Dirichlet* boundary conditions, an additional constraint on pressure correction is needed. We define the velocity and the pressure space as follows:

$$V_0 := H_0^1(\Omega), \quad V_D := \left\{ \mathbf{v} \in H^1(\Omega) \mid \mathbf{v}|_{\Gamma^{up}} = \mathbf{i}, \mathbf{v}|_{\partial\Omega \setminus \Gamma^{up}} = \mathbf{0} \right\}, \quad Q := L_0^2(\Omega).$$

and we introduce the lifting $\mathcal{R}\mathbf{g} \in V_D$, where $\mathbf{g} : \partial\Omega \rightarrow \mathbb{R}^2$ denotes the boundary data over the whole of $\partial\Omega$. Testing the system against test functions and integrating by parts, we obtain the following weak problem:

$$\begin{aligned} &\text{Find } \mathbf{u} = \tilde{\mathbf{u}} + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \tilde{\mathbf{u}} \in V_0, \text{ and } p \in Q \text{ s.t.} \\ &a(\tilde{\mathbf{u}}, \mathbf{v}) + b(\tilde{\mathbf{u}}, q) + b(\mathbf{v}, p) = -a(\mathcal{R}\mathbf{g}, \mathbf{v}) - b(\mathcal{R}\mathbf{g}, q) \quad \text{for all } (\mathbf{v}, q) \in V_0 \times Q, \\ &\text{where} \quad a(\mathbf{u}, \mathbf{v}) := \int_{\Omega} (\nabla \mathbf{u} : \nabla \mathbf{v}) \, d\mathbf{x}, \quad b(\mathbf{v}, q) := - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, d\mathbf{x}. \end{aligned}$$

Introducing the bases $\{\varphi_i\}_{i=1}^{N_V}$ and $\{\psi_\ell\}_{\ell=1}^{N_Q}$ for the FE discrete spaces $V_{0,h} \subset V_0$ and $Q_h \subset Q$, respectively, the following matrices and vectors can be defined:

$$\begin{aligned} A_{ij} &= a(\varphi_j, \varphi_i), & B_{\ell i} &= b(\varphi_i, \psi_\ell), & \mathbf{f}_i &= -a(\mathcal{R}\mathbf{g}, \varphi_i) & \mathbf{f}_{\operatorname{div}, \ell} &= -b(\mathcal{R}\mathbf{g}, \psi_\ell) \\ && \text{for } i, j &= 1, \dots, N_V, & \ell &= 1, \dots, N_Q. \\ \Sigma &= \begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} \in \mathbb{R}^{(N_V+N_Q) \times (N_V+N_Q)}, & \mathbf{F} &= \begin{bmatrix} \mathbf{f} \\ \mathbf{f}_{\operatorname{div}} \end{bmatrix} \in \mathbb{R}^{(N_V+N_Q)}, \end{aligned}$$

¹See <https://petsc.org/main/manual/ksp/> for further information. In this course we cover the main functionalities of PETSc, but there are many more!

and the Finite Element Method for the approximation of the problem can be expressed in the following algebraic form:

$$\text{find } \mathbf{U} \in \mathbb{R}^{N_V}, \mathbf{P} \in \mathbb{R}^{N_Q} \quad \text{such that} \quad \Sigma \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \end{bmatrix} = \mathbf{F}.$$

N.B. Notice that, if $\mathcal{R}g$ is such that $\text{div } \mathcal{R}g = 0$, then $\mathbf{f}_{\text{div}} = \mathbf{0}$ and the continuity equation is unperturbed. From now on, we will always make the assumption that the lifting is divergence-free.

To assemble the system block matrix, we proceed as in the previous lab session, defining the finite element spaces ($\mathbf{V}$, $\mathbf{Q}$, and the mixed space $\mathbf{W}$) and functions over a `UnitSquareMesh`. To account for pressure non-uniqueness in the fully Dirichlet case, we also create the null space of the system matrix Σ :

```
nsp = MixedVectorSpaceBasis(
    W, [W.sub(0), VectorSpaceBasis(constant=True)]
)
```

We then assemble the problem by passing the forms and boundary conditions to `LinearVariationalProblem`:

```
wh = Function(W)
vpb = LinearVariationalProblem(a, L, wh, bcs)
```

- 2 To solve the problem with the prescribed parameters, we create a `LinearVariationalSolver` including the problem and the null space created at the previous point:

```
parameters = {'ksp_type': 'gmres', 'pc_type': 'none',
              'ksp_rtol': 1.e-8, 'ksp_max_it': 10000}
solver = LinearVariationalSolver(vpb, nullspace=nsp,
                                solver_parameters=parameters)
```

Then we solve the problem, and in order to investigate the performances of our solver we inspect both the number of iterations and actual computational time required for the solution. The former is accessible from the `solver` via its underlying PETSc “engine” `solver.snes.ksp`, while for the latter we need to import the `time` python module and use its `perf_counter`, which stores the current time at which we are when it is called:

```
from time import perf_counter
t0 = perf_counter()
solver.solve()
print('elapsed time = ', perf_counter() - t0, 's' - # iter = ', \
      solver.snes.ksp.getIterationNumber())
```

Solving the problem with $n = 10$, we notice that a very large number of linear solver iterations (9054) are required, and for finer meshes 10000 iterations are not enough to

preconditioner	$n = 10$	$n = 20$	$n = 40$	$n = 80$
none	9054	–	–	–
iLU	40	118	383	–

Table 1: GMRES iterations required to solve the problem with no preconditioning and iLU preconditioning (not reported if exceeding 10'000).

reach convergence.² Indeed, the cavity problem has a strong singularity in the pressure distribution, therefore its solution can be very expensive if approached naively.

Note that the fact that convergence is not reached by the linear solver is signaled by the following error message:

```
ConvergenceError: Nonlinear solve failed to converge after 0
    nonlinear iterations.
Reason:
    DIVERGED_LINEAR_SOLVE
```

Starting from the bottom, the message **DIVERGED_LINEAR_SOLVE** should not be interpreted as the solution having *diverged* (namely reached values close to the maximum representable numbers in the machine, or contained **NaN** values), but rather as the solver **not reaching convergence** in the prescribed maximum number of iterations. Regarding the first line of the error, we must mention that for Firedrake every problem is solved in its *nonlinear solver* framework, which however runs for 0 **nonlinear iterations** when it recognizes that the problem is actually linear.

Inspired by possible previous knowledge on elliptic problems, one could think to address this issue by employing preconditioning in the iterative solver, applying the pre-defined preconditioners of PETSc on the monolithic matrix Σ (see <https://petsc.org/main/manual/ksp/#sec-pc> for a list of PETSc pre-defined preconditioners). For example, we could use an incomplete LU factorization as a preconditioner, by changing the parameter **pc_type** as follows:

```
parameters.update({'pc_type': 'ilu'})
solver = LinearVariationalSolver(vpb, nullspace=nsp,
    solver_parameters=parameters)
```

This choice improves the computational efficiency of the solution, but is not enough to solve the problem for increasingly refined mesh, as shown in Table 1. In the next lab, we will see how to implement more effective and efficient preconditioners.

²The number of iterations that the solution takes can vary slightly from one computer to another, especially in the case of ill-conditioned linear systems, due to the slight differences in the generation of machine-precision errors. The computational time measured by **perf_counter** is even more fluctuating, since it is highly sensitive to the current load of the computer. Yet, all comments reported here hold in any case.

Exercise 2

- 1 By computing the derivatives of $\mathbf{u}$ and p , we obtain

$$\mathbf{f}(x, y) = \begin{bmatrix} -2 \cos x \sin y + \frac{1}{2} \sin(2x) \\ 2 \cos x \sin y + \frac{1}{2} \sin(2y) \end{bmatrix}$$

while for $\mathbf{g}$ we are simply going to use $\mathbf{g} = \mathbf{u}|_{\partial\Omega}$.

- 2 The weak formulation of this problem reads as follows:

$$\begin{aligned} &\text{Find } \mathbf{u} = \tilde{\mathbf{u}} + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \tilde{\mathbf{u}} \in V_0, \text{ and } p \in Q \text{ s.t.} \\ &a((\tilde{\mathbf{u}}, p), (\mathbf{v}, q)) = (\mathbf{f}, \mathbf{v})_{L^2} - a((\mathcal{R}\mathbf{g}, 0), (\mathbf{v}, q)), \text{ for all } (\mathbf{v}, q) \in V_0 \times Q, \end{aligned}$$

where

$$\begin{aligned} V_D &= \{\mathbf{v} \in [H^1(\Omega)]^2 : \mathbf{v} = \mathbf{g} \text{ on } \partial\Omega\}, & V_0 &= [H_0^1(\Omega)]^2, & Q &= L_0^2(\Omega), \\ a((\mathbf{u}, p), (\mathbf{v}, q)) &:= \int_{\Omega} (\nabla \mathbf{u} : \nabla \mathbf{v} - p \operatorname{div} \mathbf{v} + \operatorname{div} \mathbf{u} \, q) \, d\mathbf{x}. \end{aligned}$$

- 3 The associated Firedrake implementation is analogous to that of Exercise 1. To avoid code modification and speed up the implementation of the convergence tests (see next points), we wrap everything in a function that, given the number $\mathbf{n}$ of mesh subdivisions along each direction, the FE polynomial degree for velocity and pressure, and two *generic* functions (see below) for $\mathbf{g}$ and $\mathbf{f}$, it returns the FE solution of the problem and the mesh (remember: in Python, indentation is part of the code!):

```
def solve_stokes(n, degreeU, degreeP, gD_fun, f_fun):
    mesh = UnitSquareMesh(n, n, 'crossed')
    ...
    # The data generic expression are interpolated on the current mesh.
    x = SpatialCoordinate(mesh)
    gD = gD_fun(x)
    f = f_fun(x)
    ...
    wh = Function(W)
    solve(a == L, wh, bcs=bc)
    uh, ph = w.split()

    return uh, ph, mesh
```

Then, outside of this function, we define the exact solution and the source term as python `lambda` functions, using the `as_vector` command for vectorial functions:

```
u_ex_fun = lambda x: as_vector((
    -cos(x[0]) * sin(x[1]),
    sin(x[0]) * cos(x[1])))
p_ex_fun = lambda x: -0.25 * (cos(2*x[0]) + cos(2*x[1])) + 0.25*sin(2.0)
f_fun = lambda x: as_vector((
```

n	N_{dofs}	$\ \mathbf{u} - \mathbf{u}_h\ _{L^2}$	$\ \mathbf{u} - \mathbf{u}_h\ _{H^1}$	$\ p - p_h\ _{L^2}$	β_{L^2}	β_{H^1}
10	1003	$1.656 \cdot 10^{-5}$	$1.262 \cdot 10^{-3}$	109	-	-
20	3803	$2.070 \cdot 10^{-6}$	$3.154 \cdot 10^{-4}$	215	3.001	2.001
40	14803	$2.587 \cdot 10^{-7}$	$7.884 \cdot 10^{-5}$	1436	3.000	2.000

n	N_{dofs}	$\ \mathbf{u} - \mathbf{u}_h\ _{L^2}$	$\ \mathbf{u} - \mathbf{u}_h\ _{H^1}$	$\ p - p_h\ _{L^2}$	β_{L^2}	β_{H^1}
10	763	$7.380 \cdot 10^{-4}$	$3.170 \cdot 10^{-2}$	354	-	-
20	2923	$1.829 \cdot 10^{-4}$	$1.563 \cdot 10^{-2}$	1439	2.013	1.020
40	11443	$4.562 \cdot 10^{-5}$	$7.760 \cdot 10^{-3}$	58761	2.003	1.010

Table 2: Convergence error and order estimates, for the FE pairs $\mathbb{P}^2/\mathbb{P}^1$ (top) and $\mathbb{P}_b^1/\mathbb{P}^1$ (bottom).

```
-2 * cos(x[0]) * sin(x[1]) + 0.5 * sin(2 * x[0]),
2 * sin(x[0]) * cos(x[1]) + 0.5 * sin(2 * x[1]))
```

Then, the convergence loop boils down to repeated calls of `solve_stokes` with different mesh subdivision numbers `n`:

```
n_vec = np.array((10, 20, 40))
# n_vec.shape -> [# rows, # columns, ...]
err_u_L2 = np.zeros(n_vec.shape[0])
err_u_H1 = np.zeros(n_vec.shape[0])
err_p_L2 = np.zeros(n_vec.shape[0])

for ii in range(n_vec.shape[0]):
    n = n_vec[ii]
    uh, ph, mesh = solve_stokes(n, 2, 1, u_ex_fun, f_fun)

    # Interpolate lambda functions on current mesh
    x = SpatialCoordinate(mesh)
    u_ex = u_ex_fun(x)
    p_ex = p_ex_fun(x)

    err_u_L2[ii] = errornorm(u_ex, uh, 'L2')
    err_u_H1[ii] = errornorm(u_ex, uh, 'H1')
    err_p_L2[ii] = errornorm(p_ex, ph, 'L2')
```

The results are summarized in Table 2.

Remark The results you obtain may be different than those reported in Table 2 and Figure 1, in particular regarding the trend of the error trend in the first implementation with NO pressure correction. Specifically, such NO-correction error may be monotonically decreasing with reducing $h...$ but still it will not have the order expected from the theoretical error estimates!

A distinction between the behaviors of the velocity errors and the pressure errors clearly shows up. Focusing on the velocity error, we notice that it decreases when a finer mesh is considered. Moreover, one can show that this decreasing follows a power law with respect

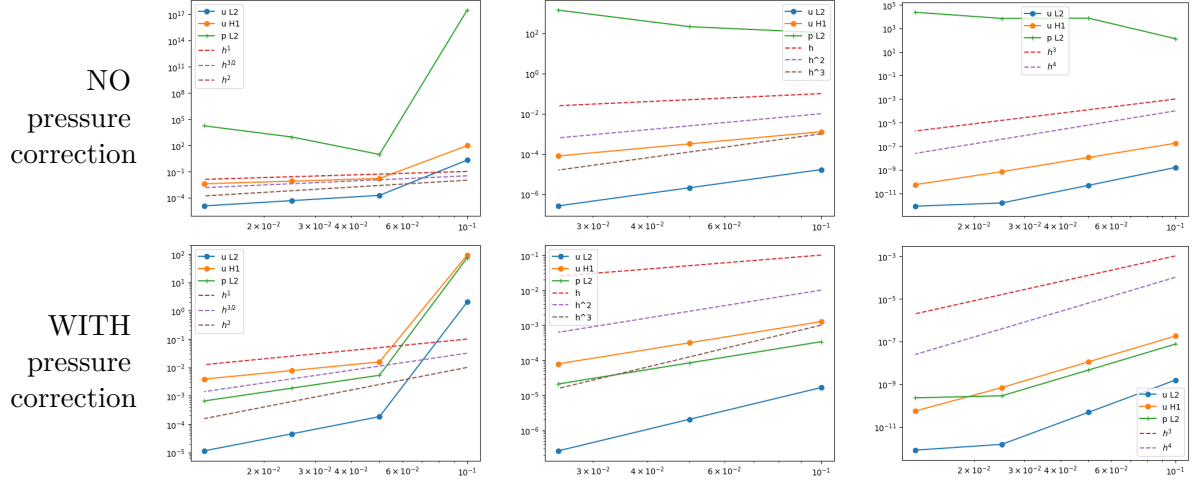


Figure 1: Convergence plots of Exercise 2: $\mathbb{P}_b^1/\mathbb{P}^1$, $\mathbb{P}^2/\mathbb{P}^1$, $\mathbb{P}^4/\mathbb{P}^3$, on the left, center, right, respectively.

to the mesh spacing $h = 1/n$, that can be proven to be the following:

$$\begin{aligned} \|\mathbf{u} - \mathbf{u}_h\|_{L^2} &= \begin{cases} \mathcal{O}(h^{m+1}), & \text{for } \mathbb{P}^m/\mathbb{P}^{m-1} \\ \mathcal{O}(h^2), & \text{for } \mathbb{P}_b^1/\mathbb{P}^1 \end{cases} \\ \|\nabla \mathbf{u} - \nabla \mathbf{u}_h\|_{L^2} &= \begin{cases} \mathcal{O}(h^m), & \text{for } \mathbb{P}^m/\mathbb{P}^{m-1} \\ \mathcal{O}(h), & \text{for } \mathbb{P}_b^1/\mathbb{P}^1 \end{cases} \end{aligned} \quad (1)$$

This can be seen by the plots reported in Figure 1 and obtained by the following lines:

```
h_vec = list(1./el for el in n_vec)
plt.loglog(h_vec, err_u_L2, marker='o', label='u L2')
plt.loglog(h_vec, err_u_H1, marker='o', label='u H1')
plt.loglog(h_vec, err_p_L2, marker='+', label='p L2')
plt.loglog(h_vec, h_vec, linestyle='--', label='h')
plt.loglog(h_vec, list(h**2 for h in h_vec), linestyle='--')
plt.loglog(h_vec, list(h**3 for h in h_vec), linestyle='--')
plt.legend()
```

Algebraically, we can estimate the order by the following formula, holding for any two values h_1, h_2 of h :

$$f(h) = h^\beta \quad \Leftrightarrow \quad \frac{f(h_1)}{f(h_2)} = \left(\frac{h_1}{h_2}\right)^\beta \quad \Leftrightarrow \quad \beta = \frac{\log(f(h_1)/f(h_2))}{\log(h_1/h_2)}.$$

For every $n = 10, 20, 40$, we define

$$\beta_{L^2} = \frac{\log(\|\mathbf{u} - \mathbf{u}_h(h = 1/n)\|_{L^2} / \|\mathbf{u} - \mathbf{u}_h(h = 2/n)\|_{L^2})}{\log(1/2)},$$

and β_{H^1} is defined analogously. The order estimates are reported in Table 2: we can notice the agreement with the orders reported in (1).

On the other hand, regarding pressure, no monotonic trend of the error can be observed in Table 2 and Figure 1 - top, for the different mesh sizes considered (actually, the pressure error increases for finer meshes!). Indeed, since fully Dirichlet boundary conditions are considered, the choice of $L^2(\Omega)$ and pressure is defined up to an additive constant even if stable FE pairs are considered. At the algebraic level, this means that the monolithic system matrix

$$\Sigma = \begin{bmatrix} A & B' \\ B & 0 \end{bmatrix}$$

has a non-trivial null space, namely $\text{Ker}(\Sigma) \neq \{\mathbf{0}\}$.

To solve this issue, one should set the problem in the space $L_0^2(\Omega) = \{p \in L^2(\Omega) \mid \int_{\Omega} p \, d\mathbf{x} = 0\}$, or other analogous subspaces of $L^2(\Omega)$. Instead of doing this, in Firedrake we can exploit the properties of Krylov linear solvers (such as the GMRES method) and “instruct” the linear solver to take into account the non-trivial null space of Σ . In the `solve_stokes` function, we generate a basis for $\text{Ker}(\Sigma)$, which is made of constant functions in the pressure component (passing the velocity subspace `W.sub(0)` as a first component means that we are skipping the velocity subspace basis, see [here](#)) and then we pass it to the solver:

```

nullsp = MixedVectorSpaceBasis(
    W, [W.sub(0), VectorSpaceBasis(constant=True)]
)
wh = Function(W)
solve(a == L, wh, bcs=bc, nullspace=nullsp)

```

This correction fixes the indetermination of pressure in the fully Dirichlet case, and convergence tests show that the pressure error is now decreasing with h . However, this is not enough to recover the correct convergence order, due to approximations in the computation and use of the basis that are of lower order w.r.t. the pressure error. This issue is addressed in the next point.

- 4 To obtain the correct convergence order in pressure we use the correction suggested in the text of the exercise, and compute the error of $\hat{p}_h = p - \frac{1}{|\Omega|} \int_{\Omega} p_h \, d\mathbf{x}$, as follows:

```

# instead of err_p_L2[ii] = errornorm(p_ex, ph, 'L2')
mean_ph = assemble(ph*dx)
err_p_L2[ii] = sqrt(assemble(
    (ph-mean_ph - p_ex)*(ph-mean_ph - p_ex)*dx))

```

The results are reported in Figure 1 - bottom, where the expected orders w.r.t. h can be observed:

$$\|p - p_h\|_{L^2} = \begin{cases} \mathcal{O}(h^m), & \text{for } \mathbb{P}^m/\mathbb{P}^{m-1}, \\ \mathcal{O}(h^{3/2}), & \text{for } \mathbb{P}_b^1/\mathbb{P}^1. \end{cases}$$

Notice that in the $\mathbb{P}_b^1/\mathbb{P}^1$ case, the error does not exhibit the expected order of convergence for “large” values of h : this is due to the fact that the theoretical estimate are valid only in the *asymptotic regime* for $h \rightarrow 0$, so they do not cover the range of large values of h . On the other hand, in the $\mathbb{P}^4/\mathbb{P}^3$ case, the errors plateau for *small* values of h . This is, instead, due to the fact that the results are affected by floating-point errors, that dominate the

FE approximation error when h is very small: indeed, the effect of this round-off errors is appreciable on values around $10^{-9} - 10^{-12}$.

Further investigation on the null space of the problem matrix can be carried out assembling it and inspecting its spectrum. The full analysis is reported in `CFDlab02.ipynb`: here we comment the main outcome. Given the bilinear form `a` and the boundary conditions `bcs` of the problem, we can assemble a matrix of tipe `firedrake.Matrix` by

```
Sigma = assemble(a, bcs=bcs)
```

This object, encoding all the features of matrices needed for assembling and solving algebraic systems in Firedrake, is not however suitable for a spectrum investigation. To do that, we need to extract the “raw data” stored in it, by

```
Sigma = assemble(a, bcs=bcs).M.values
```

where the `.M` attribute returns the PETSc `Mat` type that is actually used in the linear solvers (we remind that Firedrake uses PETSc as a linear algebra back-end), and `.values` extracts the elements of the matrix into a NumPy array. Using the `linalg` module of NumPy, we have access to many capabilities like computing the eigenvalues and eigenvectors (`np.linalg.eig`), from which we can extract the 0 eigenvalue corresponding to the null space and the corresponding eigenmode (i.e. eigenvector function). We notice that only 1 eigenvalue is equal to zero (up to machine precision) and that the corresponding eigenmode is a constant value for the pressure component: this is consistent with the theory in the case of *fully* Dirichlet boundary conditions, from which we know that pressure is defined up to an additive constant, i.e. $Ker(\Sigma)$ is made of solutions with constant pressure.